Summary in Graph

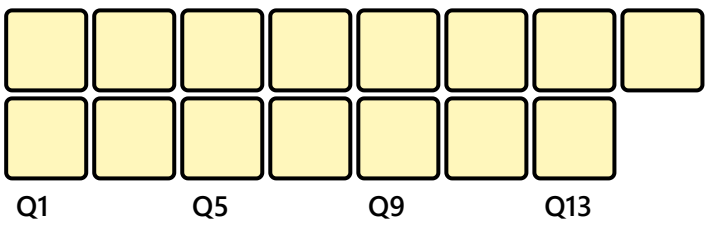
Exam Summary (GO Classes GATE CS | Operating System | Deadlock | Weekly Quiz 3)

Qs. Attempted:	0 (0 + 0)	Correct Marks:	0 (0 + 0)
Correct Attempts:	0 (0 + 0)	Penalty Marks:	0 (0 + 0)
Incorrect Attempts:	0 (0 + 0)	Resultant Marks:	0 (0 + 0)

Total Questions:	15 (10 + 5)
Total Marks:	20 (10 + 10)
Exam Duration:	45 Minutes
Time Taken:	0 Minutes

Individual Question Statistics

Select Metric: Marks Obtained



* Average Stats based on first-attempt stats of Top 10% (16) out of 159 users

EXAM RESPONSE EXAM STATS FEEDBACK

Technical

Q #1 Multiple Select Type Award: 1 Penalty: 0 Operating System

Which of the following are true about deadlocks? (choose all that apply):

A. Acquiring resources in a predetermined order is one way to avoid deadlock.

B. Rolling back threads is one way to address deadlock once it occurs.

C. Deadlock means that no threads are able to make progress.

D. Banker's algorithm is one way to recover from deadlock once it occurs.

Your Answer: | Correct Answer: A;B | Not Attempted | Time taken: 00min 00sec | Discuss

Q #2 | Multiple Select Type | Award: 1 | Penalty: 0 | Operating System

Which of the following are true regarding Banker's Algorithm?

A. It only grants resource requests that will keep the system in a "safe" state so that it will have the potential to complete without deadlock.

B. It can always find an allocation of resources that avoids deadlock

C. If it detects an "unsafe" state, then the system is guaranteed to deadlock

D. It is capable of handling threads whose maximum possible resource consumption for a particular resource changes over time

Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 00sec | Discuss

Q #3 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Operating System

Consider a system with three threads T_1, T_2, T_3 and three mutex locks x, y, z. Each thread executes the following pseudo-code:

Thread T_1	**Thread T_2**	**Thread T_3**
acquire(y)	acquire(z)	acquire(x)
acquire(z)	acquire(x)	acquire(y)
release(y)	release(z)	release(x)
release(z)	release(x)	release(y)

All threads execute concurrently.

Which of the following sequences of lock acquisitions can lead to a deadlock?

A. $T_1 : \text{acquire(y)}, T_2 : \text{acquire(z)}, T_3 : \text{acquire(x)}, T_1 : \text{acquire(x)}$

B. $T_1 : \text{acquire(y)}, T_2 : \text{acquire(z)}, T_3 : \text{acquire(x)}, T_1 : \text{acquire(z)}$

C. $T_1 : \text{acquire(z)}, T_2 : \text{acquire(y)}, T_3 : \text{acquire(x)}, T_3 : \text{acquire(y)}$

D. $T_1 : \text{acquire(y)}, T_2 : \text{acquire(z)}, T_3 : \text{acquire(x)}, T_2 : \text{acquire(x)}, T_3 : \text{acquire(y)}$

Your Answer: | Correct Answer: D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #4 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Operating System

Consider the same system with threads T_1, T_2, T_3 defined as follows:

Thread T_1	**Thread T_2**	**Thread T_3**
$\text{acquire}(y)$	$\text{acquire}(z)$	$\text{acquire}(x)$
$\text{acquire}(z)$	$\text{acquire}(x)$	$\text{acquire}(y)$
$\text{release}(y)$	$\text{release}(z)$	$\text{release}(x)$
$\text{release}(z)$	$\text{release}(x)$	$\text{release}(y)$

The threads start simultaneously. Identify the point when the system first enters an **unsafe state**.

A. After T_1 acquires y

B. After T_1, T_2, T_3 each acquire one distinct lock

C. After T_2 acquires x

D. After T_3 releases x

Your Answer: | Correct Answer: B | Not Attempted | Time taken: 00min 00sec | Discuss

Q #5 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Operating System

Consider the same system with threads T_1, T_2, T_3 defined as follows:

Thread T_1	**Thread T_2**	**Thread T_3**
$\text{acquire}(y)$	$\text{acquire}(z)$	$\text{acquire}(x)$
$\text{acquire}(z)$	$\text{acquire}(x)$	$\text{acquire}(y)$
$\text{release}(y)$	$\text{release}(z)$	$\text{release}(x)$
$\text{release}(z)$	$\text{release}(x)$	$\text{release}(y)$

Initially, the program may experience deadlock due to circular waiting between these threads and T_3. To fix this, the student modifies T_3.
Which of the following implementations of T_3 will make the program deadlock-free regardless of the scheduler?

A. $T_3 : \text{acquire(x); acquire(y); release(x); release(y)}$

B. $T_3 : \text{acquire(y); acquire(x); release(x); release(y)}$

C. $T_3 : \text{acquire}(z); \text{acquire}(x); \text{release}(z); \text{release}(x)$

D. $T_3 : \text{acquire(y); acquire(z); release(z); release(y)}$

Your Answer: | Correct Answer: B | Not Attempted | Time taken: 00min 01sec | Discuss

Q #6 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Operating System

In a system, each lock is a 2-mutex, meaning up to two threads can hold the same lock simultaneously.
A third thread attempting to acquire the same lock will block until one of the two holders releases it.
Consider two 2-mutexes: $\mathbf{a}$ and $\mathbf{b}$, and three threads: $\mathbf{x}, \mathbf{y}$, and $\mathbf{z}$.
The following sequence of lock acquisitions occurs:

1. Thread $\mathbf{z}$ acquires $\mathbf{a}$ and $\mathbf{b}$.

2. Thread $\mathbf{x}$ acquires $\mathbf{a}$.

3. Thread $\mathbf{y}$ acquires $\mathbf{b}$.

4. Thread $\mathbf{x}$ tries to acquire $\mathbf{b}$ (blocks).

5. Thread $\mathbf{y}$ tries to acquire $\mathbf{a}$ (blocks).

At this point, there is a cycle in the wait-for graph (edges from $x \rightarrow y$ and $y \rightarrow x$).
Which of the following statements is TRUE regarding this situation?

A. The system is in deadlock, since a cycle always implies circular waiting.

B. The system is not in deadlock because $\mathbf{z}$ can release its locks and allow progress.

C. The system will remain blocked indefinitely, even if $\mathbf{z}$ releases its locks.

D. The scheduler must preempt z to ensure progress of x and y.

Your Answer: | Correct Answer: B | Not Attempted | Time taken: 00min 00sec | Discuss

Q #7 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Operating System

Consider a program with two threads T_1 (Pikachu) and T_2 (Mew), both using two mutex locks : morph and levelup. The pseudo-code executed by each thread is shown below:

Thread T_1: Pikachu()	**Thread T_2: Mew()**
$\text{lock}(\text{morph})$	$\text{lock}(\text{levelup})$
$\text{lock}(\text{levelup})$	$\text{lock}(\text{morph})$
$\text{store some electricity}$	become invisible
$\text{unlock}(\text{levelup})$	$\text{unlock}(\text{levelup})$
$\text{unlock}(\text{morph})$	$\text{unlock}(\text{morph})$

Both threads start concurrently. Assume that mutex locks are **standard and cannot be acquired again by the same thread before being released**.

Which of the following statements is <u>TRUE</u> about the behavior of this program?

A. The program is deadlock-free because both threads eventually release all locks.

B. The program may deadlock if T_1 acquires `morph` and T_2 acquires `levelup` simultaneously, and preemption cannot resolve it.

C. The program may deadlock, but preemptive scheduling can break it by switching threads.

D. Deadlock cannot occur because both threads unlock all resources before exiting.

Your Answer: | Correct Answer: B | Not Attempted | Time taken: 00min 00sec | Discuss

Q #8 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Operating System

Choose all methods that can help a system avoid or prevent deadlocks:

A. Allocate virtually "infinite" (i.e., a very large number of) resources that can be shared among threads.

B. Dynamically delay resource requests that would otherwise put the system in an unsafe state.

C. Request resources in a fixed order across all threads.

D. All of the above.

Your Answer: | Correct Answer: D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #9 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Operating System

In the context of the Banker's Algorithm, which of the following best defines a $\mathbf{SAFE\ state}$?

A. A state where all processes have been allocated their maximum required resources.

B. A state where no process is waiting for any resource at the moment.

C. A state where there exists at least one sequence in which all processes can complete execution without causing deadlock, without forcing any process to release resources prematurely, and while staying within their maximum declared needs.

D. A state that is not currently deadlocked but may lead to deadlock depending on future requests.

Your Answer: | Correct Answer: C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #10 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Operating System

A system has $\mathbf{12\ drives}$ in total.
Process A starts with $\mathbf{0\ drives}$,
Process B starts with $\mathbf{4\ drives}$.
Process A first requests $\mathbf{3drives}$, and then later requests $\mathbf{x}$ more drives, where $\mathbf{x > 5}$ and $\mathbf{x \leq 9}$.
Process B may also randomly request more drives (if `rand_bool()` returns true).
The system uses the \textbf{Banker's Algorithm} to avoid deadlock.
What will the Banker's Algorithm do when \textbf{Process A} makes its second request (for $\mathbf{x}$ drives)?

```
void process_A() {
    request_more_drives(3);
    request_more_drives(x); // where x > 5 and x <= 9
    free_all_drives();
}

void process_B() {
    if (rand_bool()) { // randomly true or false
        request_more_drives(i); // where 5 < i < 9
    }
    free_all_drives();
}
```

A. The first request in `thread_A` (for 3 drives) would be denied because it causes an unsafe state.

B. The second request in `thread_A` (for x drives) would be denied because granting it could lead to an unsafe state, even though the system might still run to completion without deadlock.

C. The system will always be safe regardless of the value of `rand_bool()`.

D. The system will immediately enter a deadlock after `thread_B` calls `free_all_drives()`.

Your Answer: | Correct Answer: B | Not Attempted | Time taken: 00min 00sec | Discuss

Q #11 | Multiple Select Type | Award: 2 | Penalty: 0 | Operating System

Consider a system that manages **three resource types: A, B**, and **C**.
The total number of available instances of each resource type is given as:

$$A_{\text{total}} = 12, \quad B_{\text{total}} = 9, \quad C_{\text{total}} = 12$$

Four threads (T1, T2, T3, and T4) have the following current allocations and maximum requirements:

Thread	Current Allocation (A, B, C)	Maximum (A, B, C)
$T1$	$(2, 1, 3)$	$(4, 9, 4)$
$T2$	$(1, 2, 3)$	$(5, 3, 3)$
$T3$	$(5, 4, 3)$	$(6, 4, 3)$
$T4$	$(2, 1, 2)$	$(4, 8, 2)$

What can we conclude about the system's current state according to the **Banker's Algorithm**? (There may be multiple correct answers.)

A. The system is in an unsafe state; no safe sequence exists.

B. The system is in a safe state with possible sequence $T3 \rightarrow T2 \rightarrow T4 \rightarrow T1$.

C. The system will immediately deadlock if $T1$ makes any additional resource request, even if it does not request its full maximum capacity at once.

D. The system can remain safe if available resources increase to $(3, 2, 2)$, allowing multiple valid sequences.

Your Answer: | Correct Answer: B;D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #12 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Operating System

A system has **3** threads ($\mathrm{T1},\ \mathrm{T2},\ \mathrm{T3}$) and **3** resource types ($\mathrm{R1}, \mathrm{R2}, \mathrm{R3}$). The current state of the system is given as:

$$\textbf{Available}: (R1, R2, R3) = (0, 2, 1)$$

Allocations

Thread	$R1$	$R2$	$R3$
$T1$	1	2	2
$T2$	5	0	1
$T3$	10	3	6

Requests

Thread	$R1$	$R2$	$R3$
$T1$	3	0	2
$T2$	0	1	1
$T3$	1	3	3

Each thread can run only if its request $\leq$ currently available resources. Once a thread completes, it releases all its allocated resources back to the system.
Which of the following represents a valid order of thread execution that allows all requests to be satisfied without deadlock?

A. $T1 \rightarrow T3 \rightarrow T2$

B. $T2 \rightarrow T1 \rightarrow T3$

C. $T3 \rightarrow T2 \rightarrow T1$

D. $T1 \rightarrow T2 \rightarrow T3$

Your Answer: | Correct Answer: B | Not Attempted | Time taken: 00min 00sec | Discuss

Q #13 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Operating System

Suppose physical memory can store $\mathbf{N}$ pages, and our demand paging system uses an LRU (Least Recently Used) replacement policy. If a process's resident set contains ($\mathbf{N+1}$) distinct pages that are accessed sequentially forever, i.e.,

$$P_1, P_2, \ldots, P_N, P_{N+1}, P_1, P_2, \ldots$$

what will be the hit rate of this demand paging system?

A. The hit rate tends to $\frac{1}{N+1}$

B. The hit rate tends to $\frac{N}{N+1}$

C. The hit rate is 0

D. The hit rate is 1

Your Answer: | Correct Answer: C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #14 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Operating System

In a multithreaded system, threads use locks to control access to shared resources. Before granting a new lock to a thread, the system runs a function **will_deadlock(lock_to_give)** to check whether giving that lock may cause a circular wait (deadlock).

```
function will_deadlock(lock_to_give) :
    start_lock ← lock_to_give
    current_lock ← lock_to_give

    while (current_lock is held by some thread) :
        holder_thread← thread holding current_lock
        if (holder_thread is waiting for another lock) :
            current_lock← lock waited by holder_thread
        else:
            break // no deadlock further on this path
        if (current_lock = start_lock) :
            _____[A]_____
            return True
    end while

    _____[B]_____
    return False
```

Choose the correct combination for [A] and [B]:

A. [A] = "Deadlock found", [B] = "Safe to give the lock"

B. [A] = "Safe to give the lock", [B] = "Deadlock found"

C. [A] = "Deadlock found", [B] = "Deadlock found"

D. [A] = "Safe to give the lock", [B] = "Safe to give the lock"

Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 00sec | Discuss

Q #15 | Multiple Select Type | Award: 2 | Penalty: 0 | Operating System

Assume that our OS uses **Banker's Algorithm** to allocate resources in order to avoid deadlock. The system contains resources $\mathbf{A}, \mathbf{B}$, and $\mathbf{C}$, and there are threads $\mathbf{1}, \mathbf{2}$, and $\mathbf{3}$. The

The total amount of resources, the current allocation, and the maximum required by each thread are as follows:

Thread	Maximum Resources Needed			Currently Allocated		
	Resource A	Resource B	Resource C	Resource A	Resource B	Resource C
Thread 1	5	3	3	1	1	1
Thread 2	4	9	6	1	1	1
Thread 3	7	3	7	1	1	1

Total Resources: A = 8, B = 9, C = 10

At this point, **Thread 1** requests all the remaining resources it needs, and Banker's Algorithm must decide whether this request should be granted.
However, the algorithm is **buggy**, whenever it simulates Thread 3's completion, it incorrectly assumes that Thread 3 will fail to release $\mathbf{X}$ units of Resource $\mathbf{C}$ after finishing.
For what value(s) of $\mathbf{X}$ will Thread 1's original request not be granted (select all that apply)?

A. 4

B. 5

C. 6

D. 7

Your Answer: | Correct Answer: B;C;D | Not Attempted | Time taken: 00min 00sec | Discuss

Copyright & Stuff